

# Тема 8. Управление пакетами — npm

package.json, dependencies, node\_modules, package-lock.json, npm scripts и npm ci

# Цель занятия

---

- Научиться создавать Node.js-проект, описывать его через `package.json`, устанавливать пакеты и воспроизводимо восстанавливать рабочее окружение.
- Понять, что такое `npm` и из каких частей он состоит.
- Создать проект через `npm init`.
- Разобрать `package.json` как манифест проекта.
- Понять `dependencies`, `devDependencies` и `npm scripts`.
- Разобрать `node_modules` и `package-lock.json`.
- Научиться устанавливать, удалять, обновлять и проверять пакеты.
- Понять разницу между `npm install` и `npm ci`.

# Учебные вопросы

---

- 1. Что такое npm и зачем он нужен.
- 2. Инициализация проекта: `npm init`.
- 3. `package.json` — манифест Node.js-проекта.
- 4. `dependencies`, `devDependencies` и версии пакетов.
- 5. `node_modules` и `package-lock.json`.
- 6. Команды управления пакетами.
- 7. Практическая демонстрация: ESM-проект с `chalk` и `nodemon`.



# 1. Что такое npm

Пакеты, registry и CLI

# Что такое npm

---

- npm — экосистема для публикации, поиска, установки и управления JavaScript-пакетами.
- npm устанавливается вместе с Node.js.
- Пакет — это папка с кодом и файлом `package.json`.
- Пакеты могут быть библиотеками для кода или CLI-инструментами.
- npm помогает управлять зависимостями проекта.
- npm также запускает `scripts` из `package.json`.

## npm состоит из трёх частей

---

- Website: `npmjs.com` — поиск пакетов, документация, статистика, версии.
- Registry: серверное хранилище пакетов, откуда npm скачивает код.
- CLI: команда npm в терминале, через которую мы создаём проект, ставим пакеты и запускаем scripts.
- Когда мы пишем `npm install chalk`, CLI обращается к registry и скачивает пакет в проект.

# Зачем npm программисту

---

- Главная идея npm — не писать всё с нуля, а использовать готовые пакеты для типовых задач.
- Нужен HTTP-сервер или API — используем `express`.
- Нужна работа с датами — можно использовать `date-fns`.
- Нужно раскрасить вывод в консоли — используем `chalk`.
- Нужно автоматически перезапускать процесс при изменениях — используем `nodemon`.
- npm позволяет подключать такие решения как зависимости проекта.

# Роли npm в проекте

| Роль                   | Что делает                                                                    |
|------------------------|-------------------------------------------------------------------------------|
| Установщик             | Скачивает пакеты и помещает их в <code>node_modules</code>                    |
| Менеджер версий        | Записывает допустимые версии в <code>package.json</code>                      |
| Диспетчер зависимостей | Скачивает не только выбранный пакет, но и его зависимости                     |
| Запускатель scripts    | Выполняет команды из раздела <code>scripts</code> в <code>package.json</code> |

# Локальные и глобальные пакеты

---

- Локальная установка — пакет устанавливается в текущий проект.
- Локальная установка используется для зависимостей конкретного приложения.
- Глобальная установка -g делает CLI-команду доступной из разных папок.
- Для учебных и командных проектов чаще безопаснее ставить инструменты локально и запускать их через npm scripts.
- Например, nodemon лучше установить как devDependency: `npm i -D nodemon`.

# Локальная и глобальная установка

## Локально

- `npm install chalk`
- Пакет попадает в `node_modules` проекта.
- Записывается в `package.json`.
- Проект можно восстановить через `npm install` или `npm ci`.

## Глобально

- `npm install -g <package>`
- Пакет устанавливается как системная команда.
- Не описывает зависимость конкретного проекта.
- На другом компьютере его придётся устанавливать отдельно.

# Итог по npm

1

npm — это не только установка пакетов, но и инфраструктура проекта.

2

Пакет описывается через `package.json`.

3

Локальные зависимости относятся к конкретному проекту.

4

Глобальные установки подходят для отдельных CLI-инструментов, но в учебных проектах лучше опираться на `npm scripts`.

5

npm помогает воспроизводить окружение на другом компьютере.



## 2. Инициализация проекта

`npm init` и `npm init -y`

# Что делает npm init

---

- npm init создаёт package.json — манифест Node.js-проекта.
- Обычная папка превращается в описанный npm-проект.
- В package.json записываются имя, версия, entry point, scripts и другие поля.
- Позже npm будет записывать туда зависимости.
- Файл можно редактировать вручную, потому что это обычный JSON.

# Обычная инициализация

**BASH**

```
npm init
```

## Что важно

- npm задаёт вопросы о проекте.
- package name — имя пакета.
- version — версия проекта.
- description — краткое описание.
- entry point — основной файл пакета.
- author и license — служебная информация.

# Быстрая инициализация

**BASH**

```
npm init -y
```

## Что важно

- -y означает yes: принять значения по умолчанию.
- Команда сразу создаёт package.json.
- После этого файл можно открыть и отредактировать.
- Важно использовать обычный дефис: -y, не типографское тире.

# Версия проекта после npm init

---

- npm init по умолчанию часто предлагает версию 1.0.0.
- Это не значит, что любой проект обязан начинаться с 1.0.0.
- Учебный или ранний проект может иметь версию 0.1.0.
- Версия проекта хранится в поле version.
- Версии пакетов обычно описываются по правилам Semantic Versioning.

# Что делать, если ошиблись

---

- `package.json` — обычный текстовый JSON-файл.
- Его можно открыть в VS Code.
- Можно изменить `name`, `version`, `description`, `scripts` и другие поля.
- Главное — соблюдать правила JSON: двойные кавычки, запятые между полями, без запятой после последнего поля.
- Если JSON некорректный, `npm` не сможет прочитать `package.json`.

# Итог по npm init

---

- 1 npm init создаёт package.json.
- 2 npm init -y быстро создаёт package.json со значениями по умолчанию.
- 3 package.json — основа для управления зависимостями и scripts.
- 4 Файл можно редактировать вручную, но нужно соблюдать JSON-синтаксис.



## 3. package.json — манифест проекта

Метаданные, type, main и scripts

# package.json

---

- package.json — главный манифест Node.js-проекта.
- Описывает проект: имя, версия, описание, автор.
- Хранит scripts для запуска команд.
- Хранит список зависимостей.
- Может указывать тип модулей через type.
- Помогает другому разработчику восстановить окружение проекта.

# Минимальный package.json для курса

## JSON

```
{  
  "name": "demo-project",  
  "version": "1.0.0",  
  "type": "module",  
  "scripts": {  
    "start": "node index.js"  
  }  
}
```

## Что важно

- name — имя проекта.
- version — версия проекта.
- type: module включает ES Modules для .js-файлов.
- scripts.start задаёт команду запуска проекта.

# Поле type

---

- Поле type сообщает Node.js, как трактовать файлы .js в проекте.
- "type": "module" — .js файлы работают как ES Modules.
- В таком проекте используем import и export.
- require() и \_\_dirname из CommonJS напрямую не используются.
- Это согласуется с современным Express-кодом курса.

# Поле main

---

- main задаёт основной entry point пакета, когда проект подключают как библиотеку.
- main не означает, что Node автоматически запустит этот файл при открытии папки.
- Для запуска приложения обычно используют npm scripts.
- Например: "start": "node index.js".
- В обычном учебном приложении scripts важнее, чем main.
- Если проект публикуется как библиотека, main становится важнее.

# scripts

---

- `scripts` — раздел `package.json` для коротких команд проекта.
- Позволяет заменить длинные команды короткими именами.
- Команда запускается через `npm run <script>`.
- Исключения: `npm start` и `npm test` можно запускать без слова `run`.
- `scripts` делают проект понятнее для других разработчиков.

# Пример scripts

## JSON

```
{
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "test": "node --test"
  }
}
```

## Что важно

- npm start запускает node index.js.
- npm run dev запускает nodemon index.js.
- npm test запускает test-команду.
- start и test — специальные имена, для них можно не писать run.

# Как запускать scripts

| Script | Команда запуска |
|--------|-----------------|
| start  | npm start       |
| test   | npm test        |
| dev    | npm run dev     |
| build  | npm run build   |
| lint   | npm run lint    |

# Правила JSON

---

- В JSON используются двойные кавычки.
- Одинарные кавычки в JSON недопустимы.
- После последнего поля в объекте запятая не ставится.
- `package.json` — это один большой объект в фигурных скобках.
- Если структура нарушена, npm покажет ошибку чтения `package.json`.

# Итог по package.json

- 1 package.json — манифест проекта.
- 2 В новых учебных проектах курса добавляем "type": "module".
- 3 scripts задают стандартные команды запуска.

4 main — entry point пакета при подключении как библиотеки, а не универсальная команда запуска.

5 Файл должен быть валидным JSON.

## 4. Dependencies, devDependencies и версии

Какие пакеты нужны приложению и как npm записывает версии

# dependencies и devDependencies

## dependencies

- Нужны приложению во время работы.
- Пример: `express`, `pg`, `chalk`.
- Записываются при `npm install <name>`.
- Без них программа может не запуститься.

## devDependencies

- Нужны разработчику во время разработки.
- Пример: `nodemon`, `jest`, `eslint`.
- Записываются при `npm install -D <name>`.
- В production их можно исключить через `--omit=dev`.

# Установка обычной зависимости

## BASH

```
npm install chalk  
# или коротко:  
npm i chalk
```

## Что важно

- Пакет попадёт в `node_modules`.
- Запись появится в `dependencies`.
- `package-lock.json` обновится автоматически.
- Пакет можно импортировать в коде.

# Установка dev-зависимости

## BASH

```
npm install --save-dev nodemon  
# или коротко:  
npm i -D nodemon
```

## Что важно

- nodemon нужен только во время разработки.
- Пакет попадёт в devDependencies.
- Не нужно ставить его глобально.
- Проект сам описывает, какой инструмент разработки ему нужен.

# Важное уточнение про devDependencies

---

- devDependencies не означают, что npm install никогда их не установит.
- Обычный npm install в проекте обычно устанавливает и dependencies, и devDependencies.
- Для production можно использовать npm ci --omit=dev.
- Правильнее говорить: devDependencies — пакеты для разработки, а не для runtime-работы приложения.

# Semantic Versioning

---

- Semantic Versioning описывает версию как MAJOR.MINOR.PATCH.
- MAJOR — несовместимые изменения.
- MINOR — новая функциональность без нарушения совместимости.
- PATCH — исправления ошибок.
- Пример: 4.2.1.
- npm использует такие версии при записи зависимостей.

# MAJOR.MINOR.PATCH

| Часть версии  | Пример | Что означает                                |
|---------------|--------|---------------------------------------------|
| MAJOR         | 4.x.x  | Крупные изменения, возможна несовместимость |
| MINOR         | 4.2.x  | Новая совместимая функциональность          |
| PATCH         | 4.2.1  | Исправления ошибок                          |
| Полная версия | 4.2.1  | Конкретный опубликованный выпуск пакета     |

# Диапазоны версий в package.json

## JSON

```
{
  "dependencies": {
    "pkg-a": "2.4.1",
    "pkg-b": "^2.4.1",
    "pkg-c": "~2.4.1"
  }
}
```

## Что важно

- 2.4.1 — конкретная версия.
- ^2.4.1 — обычно допускает обновления внутри 2.x.
- ~2.4.1 — обычно допускает обновления внутри 2.4.x.
- Точное установленное дерево фиксируется в package-lock.json.

# package.json и package-lock.json в версиях

---

- package.json описывает намерение: какие версии допустимы.
- package-lock.json фиксирует фактическое дерево установленных версий.
- Если есть lock-файл, npm может воспроизвести то же разрешение зависимостей.
- Для строгой чистой установки по lock-файлу используется npm ci.

# Итог по dependencies

1

dependencies нужны приложению во время работы.

2

devDependencies нужны для разработки.

3

Обычный `npm install` может установить оба раздела.

4

Для production можно исключить devDependencies через `--omit=dev`.

5

Версии пакетов описываются через SemVer и диапазоны `^` / `~`.

6

`package-lock.json` фиксирует конкретное дерево версий.



# 5. node\_modules и package-lock.json

Физические файлы пакетов и воспроизводимая установка

# Что такое `node_modules`

---

- `node_modules` — папка, куда `npm` физически скачивает установленные пакеты.
- Внутри находятся выбранные пакеты и их вложенные зависимости.
- Папка быстро растёт по размеру.
- Её не редактируют вручную.
- Её не отправляют в `Git`.
- Её можно восстановить через `npm install` или `npm ci`.

# .gitignore

## TEXT

```
node_modules/  
.env
```

## Что важно

- node\_modules не отправляется в репозиторий.
- .env тоже не отправляется, потому что может содержать secrets.
- В Git должны быть package.json и package-lock.json.
- Другой разработчик восстановит node\_modules командой установки.

# Зачем нужен package-lock.json

---

- package-lock.json фиксирует конкретное разрешённое дерево зависимостей и integrity-данные.
- Содержит точные версии установленных пакетов.
- Содержит вложенные зависимости.
- Содержит integrity-значения для проверки скачанного пакета.
- Помогает воспроизводить установку на другой машине.
- Этот файл нужно коммитить в Git для приложений.

# package.json и package-lock.json

## package.json

- Редактирует разработчик и npm.
- Описывает проект.
- Хранит scripts.
- Хранит допустимые диапазоны зависимостей.

## package-lock.json

- Создаётся и обновляется npm.
- Фиксирует конкретное дерево зависимостей.
- Содержит integrity-данные.
- Нужен для воспроизводимой установки.

# Почему package-lock.json нужен в Git

---

- Без lock-файла разные машины могут получить разные разрешённые версии пакетов.
- С lock-файлом команда получает одинаковое дерево зависимостей.
- CI/CD также использует lock-файл для воспроизводимой установки.
- Поэтому передавать нужно не только package.json, но и package-lock.json.
- node\_modules при этом всё равно не передаётся.

# npm install и npm ci

## npm install

- Используется для обычной разработки.
- Может добавить новый пакет.
- Может обновить package-lock.json.
- Восстанавливает node\_modules, если папки нет.

## npm ci

- Используется для чистой установки.
- Требуется package-lock.json.
- Удаляет node\_modules и ставит заново.
- Хорош для CI/CD и проверки проекта.

# Восстановление проекта

## BASH

```
git clone <repo>  
cd <project>  
npm ci  
npm start
```

## Что важно

- npm ci ставит зависимости строго по package-lock.json.
- Команда удобна после скачивания проекта из Git.
- Если lock-файла нет, используйте npm install.
- Для CI/CD предпочтительно npm ci.

# Итог по `node_modules` и `lock`-файлу

- 1 `node_modules` — физическая папка с установленными пакетами.
- 2 `node_modules` не коммитают в Git.
- 3 `package.json` и `package-lock.json` коммитают.
- 4 `package.json` описывает проект и допустимые версии.
- 5 `package-lock.json` фиксирует конкретное дерево зависимостей.
- 6 `npm ci` нужен для чистой воспроизводимой установки.

## 6. Команды управления пакетами

install, uninstall, update, outdated, audit

# Основные команды npm

| Задача                         | Команда                                  |
|--------------------------------|------------------------------------------|
| Создать package.json           | <code>npm init -y</code>                 |
| Установить пакет               | <code>npm install &lt;name&gt;</code>    |
| Установить dev-пакет           | <code>npm install -D &lt;name&gt;</code> |
| Удалить пакет                  | <code>npm uninstall &lt;name&gt;</code>  |
| Восстановить зависимости       | <code>npm install</code>                 |
| Чистая установка по lock-файлу | <code>npm ci</code>                      |
| Запустить script               | <code>npm run &lt;script&gt;</code>      |

# Установка пакетов

---

- `npm install <name>` устанавливает пакет и записывает его в `dependencies`.
- `npm i <name>` — короткая форма.
- `npm install -D <name>` записывает пакет в `devDependencies`.
- `npm install` без имени пакета восстанавливает зависимости проекта.
- После установки обновляются `node_modules` и `package-lock.json`.

# Удаление пакетов

---

- `npm uninstall <name>` удаляет пакет из проекта.
- Пакет удаляется из `node_modules`.
- Запись удаляется из `package.json`.
- `package-lock.json` тоже обновляется.
- Не удаляйте пакет вручную только из `node_modules`: `npm` должен обновить манифест.

# Команды проверки и обновления

## BASH

```
npm outdated  
npm update  
npm list
```

## Что важно

- npm outdated показывает, какие пакеты имеют новые версии.
- npm update обновляет пакеты в пределах допустимых диапазонов package.json.
- npm list показывает дерево установленных зависимостей.
- Перед обновлением лучше понимать, какие изменения вносят новые версии.

# Проверка известных уязвимостей

## BASH

```
npm audit  
npm audit fix
```

## Что важно

- npm audit проверяет дерево зависимостей на известные проблемы безопасности.
- npm audit fix пытается автоматически применить безопасные исправления.
- npm audit fix --force может привести к major-обновлениям и поломкам.
- Не запускайте --force без понимания последствий.

# Глобальные команды

---

- `npm install -g <name>` устанавливает пакет как системную CLI-команду.
- `npm root -g` показывает путь к глобальным пакетам.
- Глобальная установка не записывает зависимость в `package.json` проекта.
- Для командной разработки лучше использовать локальные `devDependencies` и `npm scripts`.
- Так у всех участников проекта будет одинаковый инструмент.

# Итог по командам

---

1

`npm install` устанавливает зависимости и может обновлять `lock`-файл.

2

`npm ci` ставит зависимости строго по `lock`-файлу.

3

`npm uninstall` корректно удаляет пакет из проекта.

4

`npm outdated` и `npm update` помогают управлять обновлениями.

5

`npm audit` помогает проверять известные проблемы безопасности.

6

Глобальные пакеты не заменяют зависимости проекта.

# 7. Практическая демонстрация

ESM-проект с chalk и nodemon

# План демонстрации

---

- |   |                     |                                                                   |
|---|---------------------|-------------------------------------------------------------------|
| 1 | Создать папку       | <code>demo-project</code>                                         |
| 2 | Инициализировать    | <code>npm init -y</code>                                          |
| 3 | Включить ESM        | добавить <code>"type": "module"</code>                            |
| 4 | Установить chalk    | <code>npm install chalk</code>                                    |
| 5 | Написать index.js   | <code>import chalk from 'chalk'</code>                            |
| 6 | Добавить scripts    | <code>start</code> и <code>dev</code>                             |
| 7 | Восстановить проект | удалить <code>node_modules</code> и выполнить <code>npm ci</code> |

# Создаём проект

## BASH

```
mkdir demo-project  
cd demo-project  
npm init -y
```

## Что важно

- Создаём папку проекта.
- Переходим в неё.
- Создаём package.json со значениями по умолчанию.
- Дальше редактируем package.json вручную.

# Редактируем package.json

## JSON

```
{
  "name": "demo-project",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node index.js"
  }
}
```

## Что важно

- type: module включает import/export.
- start задаёт стандартную команду запуска.
- После этого можно запускать проект через npm start.
- Если package.json содержит другие поля, их можно оставить.

# Устанавливаем chalk

## BASH

```
npm install chalk
```

## Что важно

- chalk будет записан в dependencies.
- Появится или обновится package-lock.json.
- Появится папка node\_modules.
- Современный chalk используется через import.

# Используем пакет в index.js

## JAVASCRIPT

```
import chalk from 'chalk';

console.log(chalk.blue('Привет, студенты!'));
console.log(chalk.green.bold('npm работает успешно.'));
console.log(chalk.yellow('Проект использует ES Modules.'));
```

## Что важно

- Это современный ESM-синтаксис.
- `require('chalk')` здесь не используется.
- Код согласуется с `package.json`:  
"type": "module".
- Мы не писали раскраску сами, а использовали пакет из npm.

# Запуск через node и npm start

## BASH

```
node index.js
```

```
npm start
```

## Что важно

- node index.js запускает файл напрямую.
- npm start запускает script из package.json.
- Для других разработчиков npm start понятнее, чем поиск главного файла вручную.
- scripts становятся стандартной точкой входа в проект.

## Что появилось после установки

---

- `node_modules` — папка с кодом пакетов.
- `package-lock.json` — lock-файл с конкретным деревом зависимостей.
- В `package.json` появился раздел `dependencies`.
- Внутри `node_modules` находится не только `chalk`, но и его зависимости, если они нужны.
- `node_modules` не трогаем вручную и не отправляем в Git.

# Устанавливаем nodemon как devDependency

## BASH

```
npm install -D nodemon
```

## Что важно

- nodemon нужен во время разработки.
- Поэтому он попадает в devDependencies.
- Не нужно ставить его глобально.
- Проект сам описывает, какой инструмент разработки ему нужен.

# Альтернатива nodemon — встроенный watch mode Node.js

## JSON

```
{
  "scripts": {
    "start": "node index.js",
    "dev": "node --watch index.js"
  }
}
```

## Что важно

- Запуск: **npm run dev**
- изменили index.js  
↓
- сохранили файл  
↓
- node --watch заметил изменение  
↓
- Node.js автоматически перезапустил проце

# Добавляем dev script

## JSON

```
{  
  "scripts": {  
    "start": "node index.js",  
    "dev": "nodemon index.js"  
  }  
}
```

## Что важно

- `npm start` запускает обычный режим.
- `npm run dev` запускает режим разработки.
- `nodemon` перезапускает Node.js-процесс при изменении файлов.
- Важно: `nodemon` не является сервером, он только перезапускает процесс.

# Запуск режима разработки

**BASH**

```
npm run dev
```

## Что важно

- Команда запускает script dev.
- nodemon следит за изменениями файлов.
- При сохранении index.js Node.js-процесс перезапустится.
- Это удобно во время разработки.

# Фокус с исчезновением node\_modules

## BASH

```
# удалить node_modules вручную  
npm ci  
npm start
```

## Что важно

- node\_modules можно восстановить.
- npm ci использует package-lock.json.
- Именно поэтому node\_modules не передают через Git.
- В репозитории должны быть package.json и package-lock.json.

# Что отправляем в Git

## Отправляем

- package.json
- package-lock.json
- index.js
- src/
- .gitignore
- README.md

## Не отправляем

- node\_modules/
- .env
- секретные ключи
- пароли
- временные файлы

# Выводы

1

npm помогает управлять зависимостями и scripts.

2

package.json — манифест проекта.

3

Для современных учебных проектов используем "type": "module".

4

dependencies нужны приложению, devDependencies нужны для разработки.

5

Версии пакетов описываются через SemVer и диапазоны.

6

package-lock.json фиксирует конкретное дерево зависимостей.

7

node\_modules не отправляют в Git.

8

npm install удобен при разработке, npm ci — для чистой воспроизводимой установки.

9

npm scripts задают стандартные команды запуска проекта.

# Контрольные вопросы

1 Что такое npm и из каких частей он состоит?

2 Что создаёт команда `npm init -y`?

3 Зачем в `package.json` нужно поле `"type": "module"`?

4 Чем `scripts` отличаются от обычного запуска `node index.js`?

5 В чём разница между `dependencies` и `devDependencies`?

6 Почему `node_modules` не отправляют в Git?

7 Зачем нужен `package-lock.json`?

8 Чем `npm install` отличается от `npm ci`?

9 Что означает версия вида `^4.1.2`?

10 Почему `nodemon` лучше ставить как `devDependency` проекта?